West University of Timişoara
Faculty of Mathematics and Computer Science
Department of Computer Science — English Section

Sorting Algorithm Benchmark

Performance Analysis of Six Sorting Algorithms in C++

Milestone 1 — Experimental Results and Analysis

Dragos Olaru
dragos.olaru06@e-uvt.ro

Academic Year: 2025–2026
Course: Algorithmic Analysis and Design

Repository: https://github.com/dragosolaru06-creator/Academic-Writing

All experimental data produced and analysed independently by the author.

Abstract

This report presents a systematic empirical evaluation of six fundamental sorting algorithms implemented in C++, benchmarked across three element types (`int`, `float`, `char`), five input distributions, and seven input sizes ranging from $n = 20$ to $n = 1{,}000{,}000$. The study examines how algorithm behaviour deviates from theoretical predictions once hardware realities—cache efficiency, branch misprediction, and memory bandwidth—enter the picture. Key findings include: *(i)* Insertion Sort achieves near-linear performance on sorted and nearly-sorted data while deteriorating sharply on random input; *(ii)* Selection Sort performs an identical number of comparisons regardless of input order, rendering it distribution-insensitive; *(iii)* naïve Quick Sort degrades catastrophically on sorted input due to pivot-selection failure; *(iv)* Merge Sort and Heap Sort maintain predictable $\mathcal{O}(n \log n)$ scaling across all tested distributions; and *(v)* `char` arrays sort measurably faster than numeric arrays at large n due to a fourfold reduction in cache footprint. Together these results illustrate that asymptotic complexity is a necessary but insufficient predictor of real-world performance.

Contents

1. Introduction and Scope

Sorting is one of the most exhaustively studied problems in computer science and one of the most frequently executed operations in practice. Despite decades of theoretical treatment, the relationship between asymptotic complexity and observed runtime remains non-trivial: an algorithm classified as $\mathcal{O}(n^2)$ may outperform an $\mathcal{O}(n \log n)$ algorithm on small or structured inputs, while an algorithm that is theoretically optimal may be hampered by cache-unfriendly memory access patterns or branch-prediction failures at scale.

This document reports Milestone 1 of a systematic benchmarking project whose objective is precisely this gap. All implementations are written in C++ and timed using `std::chrono::high_resolution_clock`. The benchmark spans four orthogonal dimensions:

- **Algorithms (6):** Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Merge Sort, Heap Sort.
- **Element types (3):** 32-bit signed integer (`int`), IEEE 754 single-precision float (`float`), and 8-bit character (`char`).
- **Input distributions (5):** random, fully sorted, reverse-sorted, half-sorted, and nearly sorted.
- **Input sizes (7):** $n \in \{20, 50, 100, 1{,}000, 10{,}000, 100{,}000, 1{,}000{,}000\}$.

Entries where the subprocess exceeded a 60-second wall-clock limit are recorded as TIMEOUT in the raw dataset and excluded from all quantitative comparisons in this report.

2. Algorithms Under Study

Table 1 summarises the theoretical properties of each algorithm. Brief implementation notes follow.

Table 1: Theoretical properties of the six benchmarked algorithms.

Algorithm	Best	Average / Worst	Space	Stable	Adaptive	In-place
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Yes	Yes	Yes
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	Yes	Yes	Yes
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	No	No	Yes
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)^\dagger$	$\mathcal{O}(\log n)$	No	No	Yes
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	Yes	No	No
Heap Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	No	No	Yes

†Worst case occurs with naïve pivot selection (last element) on sorted/reverse-sorted input.

Bubble Sort. Optimised with an early-termination flag: if no swap is performed during a full pass, the array is already sorted and the algorithm terminates. This makes Bubble Sort $\mathcal{O}(n)$ on sorted input.

Insertion Sort. Maintains a sorted prefix; each new element is shifted left until its correct position is found. The inner loop terminates early once a smaller element is encountered, yielding linear performance on nearly-sorted data.

Selection Sort. Finds the minimum of the unsorted suffix and swaps it into position. The scan always covers the full remaining subarray, making the comparison count independent of input order.

Quick Sort. Implemented with the last element as pivot and standard Lomuto partitioning. This choice creates a $\mathcal{O}(n^2)$ worst case on sorted or reverse-sorted input, which is explicitly exposed in the benchmark.

Merge Sort. Standard top-down recursive implementation with auxiliary arrays allocated per merge call. Stable and distribution-insensitive by construction.

Heap Sort. In-place via binary max-heap. Heap construction costs $\mathcal{O}(n)$; the extraction phase costs $\mathcal{O}(n \log n)$. The traversal pattern is cache-unfriendly at large n.

3. Experimental Setup

Table 2 documents the measurement protocol.

Table 2: Benchmark parameters and measurement protocol.

Parameter	Description
Timing mechanism	`std::chrono::high_resolution_clock`; results reported in seconds (s)
Timeout policy	Each (algorithm, type, size, distribution) tuple is launched as a separate subprocess; processes exceeding 60 s are terminated and recorded as TIMEOUT
Array freshness	A fresh array is generated immediately before each timed call; the generation cost is excluded from the measurement
RNG seed	`srand(time(nullptr))` at program start; all runs share one seed sequence
Compiler	MSVC / MinGW on Windows; standard release optimisations enabled
Timeout threshold	60 seconds per configuration

All figures use logarithmic axes unless stated otherwise, because the measured time range spans more than five orders of magnitude.

4. Experimental Results

4.1 Overview: All Algorithms on Random Integer Input

Figure 1 places all six algorithms on a single log-log canvas for random `int` data. Three performance tiers are immediately visible.

Tier 1 — $\mathcal{O}(n \log n)$ cluster. Merge Sort and Heap Sort deliver consistently low runtimes across the full tested range. Their curves rise steadily with a slope close to the theoretical $\log n$ gradient.

Tier 2 — Quick Sort. Quick Sort performs comparably to Merge Sort on random data. Its near-optimal behaviour here stems from the random input providing statistically balanced partitions despite the naïve last-element pivot.

Tier 3 — Quadratic. Bubble Sort, Insertion Sort, and Selection Sort diverge sharply from $n = 1{,}000$ onward. By $n = 10{,}000$ their runtimes are two to three orders of magnitude greater than those of the efficient algorithms.

Figure 1: All six algorithms on random `int` input (log-log scale). Quadratic algorithms are measured up to $n = 1{,}000{,}000$ where the timeout threshold allows.

4.2 Quadratic Algorithms: Input Distribution Sensitivity

Figure 2 contrasts Bubble Sort and Insertion Sort across all five distributions. The difference exposes a fundamental algorithmic distinction.

Bubble Sort terminates early when no swap occurs in a pass. On a sorted array it therefore executes exactly $n - 1$ comparisons and zero swaps, reducing to a linear scan. This produces the fastest observed curve for that algorithm. On random and reverse-sorted data, however, the optimisation never fires and the full $\mathcal{O}(n^2)$ cost materialises.

Insertion Sort exhibits even more pronounced adaptivity. Its inner loop halts as soon as it encounters an element smaller than the one being inserted. On fully sorted input the inner loop never executes at all, yielding $\mathcal{O}(n)$ performance. On reverse-sorted input every element must travel the full length of the sorted prefix, producing the worst case. The data confirms this: at $n = 10{,}000$ the sorted-input time is approximately 20 times lower than the random-input time.

Figure 2: Bubble Sort (left) and Insertion Sort (right) across all input distributions on `int` data. The pronounced spread between curves reflects strong input-order sensitivity.

4.3 Selection Sort: Distribution Insensitivity

Figure 3 isolates Selection Sort. Unlike Bubble Sort and Insertion Sort, all five distribution curves lie almost exactly on top of one another throughout the tested range.

The explanation is structural. Selection Sort always scans the entire unsorted suffix to locate its minimum element before swapping it into place. There is no mechanism to

detect or exploit existing order; the comparison count for an array of size n is always $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$, regardless of how the elements are arranged. This makes Selection Sort uniquely predictable but also uniquely unresponsive to the structure of its input.

Figure 3: Selection Sort across all five input distributions (`int`). The near-perfect overlap of curves demonstrates complete distribution insensitivity.

4.4 Insertion Sort Adaptivity: A Closer Look

Figure 4 isolates the three most informative distributions for Insertion Sort: random, sorted, and nearly sorted.

On sorted input the runtime is effectively linear: the inner loop never executes more than one comparison per outer-loop iteration. On nearly-sorted input the advantage is proportionally smaller but still substantial, because the inversions that drive the inner loop are few and short. On random input the algorithm incurs the full quadratic cost.

Figure 4: Insertion Sort runtime on sorted, nearly-sorted, and random `int` input. The gap between sorted and random curves widens with n, confirming $\mathcal{O}(n)$ vs $\mathcal{O}(n^2)$ scaling.

4.5 Quick Sort: Worst-Case Exposure

Figure 5 shows Quick Sort across all five distributions. The result is among the most striking in the dataset.

On random input Quick Sort delivers excellent performance, competitive with Merge Sort. On sorted and reverse-sorted input, however, the last-element pivot selects the global minimum or maximum at every partition step, producing maximally unbalanced splits of size 1 and $n-1$. The recursion depth reaches n, and the comparison count climbs to $\mathcal{O}(n^2)$. The benchmark confirms this: at $n = 100{,}000$ the sorted-input time exceeds 40 seconds, more than 800 times the random-input time. Most configurations at $n = 1{,}000{,}000$ result in a timeout under this pivot strategy.

This result illustrates why production implementations (introsort, pdq-sort) abandon naïve pivot selection in favour of median-of-three, random pivot, or dual-pivot strategies.

Figure 5: Quick Sort (last-element pivot, Lomuto partition) across input distributions (`int`). Sorted and reverse-sorted inputs trigger the $\mathcal{O}(n^2)$ worst case at large n.

4.6 O(n log n) Algorithms Across Distributions

Figure 6 places Merge Sort, Quick Sort, and Heap Sort side by side across all five distributions. Two observations stand out.

Merge Sort is entirely distribution-insensitive. Its five curves are indistinguishable throughout the tested range; the merge procedure performs the same work regardless of input order.

Heap Sort likewise shows low sensitivity to distribution: the heap-construction and sift-down traversals do not benefit from existing order in the array. Its curves cluster tightly, though slightly above those of Merge Sort at large n— a reflection of cache-unfriendly non-sequential memory access during sift-down.

Quick Sort, as discussed above, is the outlier: distribution-sensitive in the extreme, near-optimal on random data and pathological on sorted data with naïve pivoting.

Figure 6: Quick Sort, Merge Sort, and Heap Sort across all five input distributions (`int`). Merge Sort and Heap Sort show near-flat distribution sensitivity; Quick Sort diverges sharply on ordered data.

4.7 Element Type Comparison

Figure 7 overlays `int`, `float`, and `char` results for Quick Sort and Merge Sort on random data. Figure 8 repeats the comparison for Merge Sort and Heap Sort.

Two patterns emerge consistently.

First, `float` and `int` are essentially indistinguishable across all algorithms and sizes. The overhead of IEEE 754 single-precision comparison is unmeasurable against other runtime costs on modern x86-64 hardware: the floating-point comparison unit executes in the same number of cycles as an integer comparison.

Second, `char` is measurably faster than the numeric types at large n, and the gap grows with input size. A `char` array of n elements occupies n bytes; an `int` array occupies $4n$ bytes. The fourfold reduction in memory footprint means four times as many elements fit within a single cache line. At small n the entire dataset fits in the L1 cache regardless of element size, so no advantage is visible. At $n = 100{,}000$ and beyond the cache-footprint difference becomes the dominant factor and the `char` curves separate measurably downward.

Figure 7: Runtime across element types on random input for Quick Sort and Merge Sort. `float` tracks `int` throughout; `char` diverges downward at large n due to cache-footprint savings.

Figure 8: Cache footprint effect for Merge Sort and Heap Sort on random input. The `char` advantage is visible from $n = 10{,}000$ onward and grows monotonically.

4.8 Absolute Runtime at $n = 100{,}000$

Figure 9 presents a horizontal bar chart of absolute runtimes at $n = 100{,}000$ on random `int` data, providing a direct apples-to-apples comparison.

Merge Sort and Heap Sort occupy the leading tier, separated by a margin smaller than their measurement noise. Insertion Sort, Selection Sort, and Bubble Sort are two orders of magnitude slower. Quick Sort, despite its naïve pivot, still achieves fast runtimes on random input, confirming that its pathological behaviour is distribution-dependent rather than intrinsic to the algorithm class.

Figure 9: Absolute runtime at $n = 100{,}000$, random `int` input. Bubble Sort is absent (timeout on random input at this size).

5. Summary of Key Findings

Table 3 collects the most significant quantitative results from the benchmark. All entries refer to `int` data unless otherwise stated.

Table 3: Summary of key experimental findings.

Finding	**Measured result**
Merge Sort at $n = 1{,}000{,}000$, random	0.702 s — consistent $\mathcal{O}(n \log n)$ scaling across all distributions
Quick Sort on sorted `int`, $n = 100{,}000$	40.7 s vs 0.049 s on random — factor $> 800\times$ degradation from naïve pivot selection
Insertion Sort: sorted vs random, $n = 10{,}000$	0.024 s vs 0.141 s — $\approx 6\times$ advantage on sorted data, consistent with $\mathcal{O}(n)$ vs $\mathcal{O}(n^2)$
Selection Sort: distribution sensitivity	All five distribution curves within $\pm 5\%$ of one another at every tested size — complete distribution insensitivity confirmed
`float` vs `int`	$< 3\%$ difference across all algorithms and sizes — IEEE 754 comparison overhead is negligible on modern hardware
`char` vs `int` at $n = 1{,}000{,}000$	Merge Sort: 0.500 s (`char`) vs 0.702 s (`int`) — $\approx 28\%$ faster, attributable to $4\times$ smaller cache footprint
Heap Sort on random `int`, $n = 1{,}000{,}000$	0.854 s — marginally slower than Merge Sort (0.702 s) despite lower asymptotic space usage, reflecting cache-unfriendly sift-down traversal

6. Expectations vs. Observations

Prior to running the benchmark, a set of hypotheses was formulated based on theoretical expectations. Table 4 records each hypothesis and its empirical outcome.

Table 4: Pre-experiment hypotheses and empirical outcomes.

Hypothesis	**Outcome and commentary**	**Result**
Merge Sort is the fastest algorithm at large n on random data	Confirmed: Merge Sort and Heap Sort consistently lead the field on random `int`. Quick Sort is competitive but vulnerable to distribution.	✓
Quick Sort degrades on sorted and reverse-sorted input	Confirmed: naïve last-element pivot produces $\mathcal{O}(n^2)$ behaviour on ordered data; timeout at $n = 1{,}000{,}000$ for both distributions.	✓
`float` incurs measurable comparison overhead vs `int`	Refuted: difference is $< 3\%$ throughout. Modern FPUs handle single-precision comparison in the same latency as integer comparison.	×
Selection Sort is the most distribution-sensitive $\mathcal{O}(n^2)$ algorithm	Refuted: Selection Sort is the *least* sensitive — it is Insertion Sort and Bubble Sort that exhibit dramatic distribution dependence.	×
`char` arrays sort faster than `int` arrays at large n	Confirmed: advantage appears from $n = 10{,}000$ onward and grows monotonically; cache-line packing is the dominant mechanism.	✓
Insertion Sort performs well on nearly-sorted input	Confirmed: nearly-sorted runtime is close	✓

Of six pre-stated hypotheses, four were confirmed and two refuted. Both refuted hypotheses trace to hardware effects invisible to complexity theory: IEEE 754 floating-point comparison hardware and the structural role of Selection Sort's inner loop. This reinforces the central motivation for empirical benchmarking: asymptotic analysis predicts trends, but hardware realities determine constants.

7. Conclusions and Future Work

The benchmark confirms the expected asymptotic hierarchy while revealing several distribution-dependent and hardware-mediated subtleties that theory alone does not predict.

The most practically significant finding is Quick Sort's catastrophic degradation on sorted input under naïve pivot selection. This is not a theoretical curiosity: many real-world datasets are partially sorted (databases after bulk insertion, log files, nearly-monotone time series), and a production implementation that does not address pivot selection is unsafe for general use. Milestone 2 will compare last-element pivot against median-of-three and random-pivot strategies across the same input space.

A second direction for Milestone 2 is an extended size range (n up to 10,000,000) for the $\mathcal{O}(n \log n)$ algorithms, together with profiling-based cache-miss counts to quantify the cache-footprint hypothesis for `char` arrays more rigorously.

Source code and raw dataset:
`https://github.com/dragosolaru06-creator/Academic-Writing`